



۸۱۰۱۰۱۴۰۸

۸۱۰۱۰۱۳۹۲

مبانی رایانش توزیع شده
نام و نام خانوادگی اعضای گروه:
بابک حسینی محتشم
بهراد بینایی



تمرین شماره ۱

۱ مقدمه

در این تمرین ضمن آشنایی با زبان Golang، ابتدا یک برنامه ماشین حساب می نویسیم که در دو پرده متفاوت اجرا می شود و برای کارکرد درست نیاز به ارتباط میان پرده های مناسب است، سپس با استفاده از goroutine ها که همان ریسمان ها در زبان Go می باشند تاثیر اجرای موازی برنامه را بررسی می کنیم و در نهایت یک سرور ساده را با کانتینترسازی و کمک داکر بالا می آوریم.

به صورت کلی اکثر چالش و سختی این پروژه به دلیل آشنا نبودن با زبان Golang بود ولی به دلیل سادگی این زبان در زمینه کاربردهای خواسته شده در پروژه یادگیری و انجام قسمت ها با چالشی کمی همراه بود. البته برای قسمت دوم رسم نمودارها به دلیل نیاز به نصب کتابخانه خارجی هم چالش برانگیز بود. در نهایت در قسمت سوم هم نصب و کار با داکر از آنجا که برای بار اول با آن کار می کردیم دشوار بود.

۲ بخش اول: ارتباط بین دو پردازش مستقل با Pipe

۱.۲ شرح مسئله

در این مسئله قصد داریم دو پردازش مستقل با نام‌های Interface و Worker پیاده‌سازی کنیم. پردازش Interface نقش نقطه ورود درخواست را دارد و پس از دریافت درخواست از کاربر آن را از طریق Named Pipe به پردازش Worker ارسال می‌کند. پردازش Worker با دریافت درخواست، آن را پردازش می‌کند و پاسخ را برمی‌گرداند. در نهایت پردازش Interface پاسخ را به کاربر نمایش می‌دهد.

۲.۲ پیاده‌سازی

۳.۲ Interface.go

در این فایل قصد داریم واسطی بین کاربر و Worker ایجاد کنیم. به همین منظور داده‌ساختار Response را تعریف می‌کنیم که شامل عملگر و عملوندهای داده شده توسط کاربر و نتیجه، وضعیت و ارور خروجی به دست آمده از Worker را شامل می‌شود.

انتظار داریم پردازش Worker پیش از Interface اجرا شده باشد و در آن Pipe‌های ارتباطی بین دو پردازش ایجاد شده باشند. بنابراین در ابتدا با بررسی وجود Pipe‌ها آن‌ها را باز می‌کنیم. requestPipe برای ارسال درخواست به Worker و responsePipe برای دریافت داده‌ساختار Response از Worker است.

سپس ورودی داده شده را می‌خوانیم و اگر کاربر EXIT یا QUIT وارد کرده بود پیام EXIT به معنای ختم برنامه را به Worker ارسال می‌کنیم. در غیر این صورت ورودی کاربر را از طریق پایپ برای Worker می‌فرستیم و منتظر دریافت پاسخ می‌مانیم. در نهایت با دریافت پاسخ، در تابع createResponse پاسخ را از JSON به Response تبدیل کرده و چاپ می‌کنیم.

۴.۲ Worker.go

در این فایل قصد داریم با دریافت درخواست کاربر از Interface حاصل عملیات ریاضی را حساب کرده و آن را به Interface برگردانیم. ابتدا پایپ‌ها را ایجاد می‌کنیم تا دو پردازش بتوانند با هم ارتباط برقرار کنند. سپس در صورتی که پایپ بسته نشده باشد، با دریافت ورودی بررسی می‌کنیم اگر ورودی داده شده EXIT بود، از برنامه خارج می‌شویم. وگرنه در تابع parseRequest دو عملوند آن را بررسی می‌کنیم تا از جنس عددی باشند.

در مرحله بعد تابع compute را صدا می‌زنیم و در این تابع عملیات را براساس ورودی داده شده انجام می‌دهیم و حاصل را برمی‌گردانیم. اگر عملگر داده شده معتبر نباشد، یا به تقسیم بر صفر یا پیمانه صفر برسیم ارور برمی‌گردانیم.

در نهایت در تابع `writeResponse` نتیجه را به فایل `Interface` تحویل می‌دهیم.

۵.۲ ارزیابی

برای اطمینان از کارکرد درست، هر وقت به اروری بر میخوریم یا پیامی را ارسال یا دریافت می‌کنیم، عملیات را در لاگ ثبت می‌کنیم. برای تفکیک لاگ‌ها، از رنگ قرمز برای `Worker` و رنگ آبی برای `Interface` استفاده می‌کنیم. همچنین با دادن تست‌های متنوع شامل ارورهای گوناگون از درست بودن برنامه اطمینان حاصل کردیم.

۳ بخش دوم: همزمانی، زمانبندی و مشاهده اثرات Context-Switching

۱.۳ شرح مسئله

در این بخش قصد داریم با استفاده از Goroutine ها برنامه‌ای را به صورت همروند اجرا کنیم و آن را نسبت به اجرای سریال مقایسه کنیم. یکی از اهداف طراحی زبان Go پشتیبانی از همروندی بوده است. بدین ترتیب امکانات زیادی را در عین سادگی در اختیار برنامه‌نویسان برنامه‌های موازی قرار می‌دهد. Goroutine ها مشابه ریسمان‌ها در زبان‌های دیگر هستند ولی استفاده از آن‌ها ساده‌تر است. همچنین بسیار سبک طراحی شده‌اند و به دلیل حجم کمی که می‌گیرند می‌تواند هزاران و میلیون‌ها نمونه از آن‌ها را ایجاد کرد و مدیریت آن‌ها توسط خود زبان انجام می‌شود.

۲.۳ پیاده‌سازی

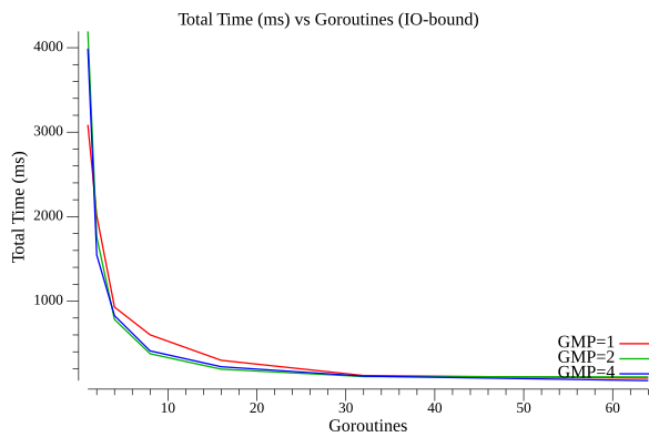
Result را تعریف می‌کنیم تا اطلاعات و نتیجه اجرای یک Goroutine را در آن ذخیره کنیم. این ساختار داده شامل تعداد تسک‌های یک Goroutine، کل زمان اجرای تمام Goroutine ها، گذرده‌ی یا نسبت تسک‌های انجام شده در واحد زمان، حداقل، حداکثر، میانگین، و انحراف معیار انجام یک تسک، تعداد Goroutine ها، نوع تسک‌ها و تعداد هسته‌های پردازشی استفاده شده است.

همان طور که خواسته شده دو نوع تسک گوناگون تعریف کردیم به طوری که تابع `cpuwork` تسک پردازشی را مشخص می‌کند که در یک حلقه طولانی یک محاسبه ریاضیاتی نسبتاً پیچیده انجام می‌شود. تابع `iowork` تسک وابسته به I/O را نشان می‌دهد که پس از محاسبات کوتاه به مدت ۳ میلی‌ثانیه به `sleep` می‌رود.

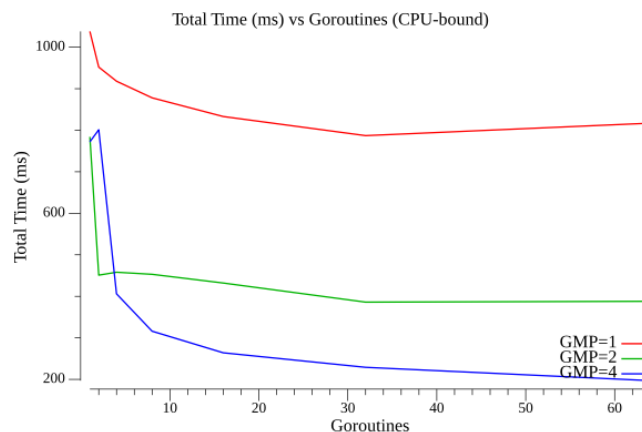
تابع `runBenchmark` با دریافت تعداد Goroutine ها، نوع تسک و تعداد هسته‌های پردازشی پردازنده در تابع اصلی فراخوانی می‌شود. تعداد کل تسک‌ها برابر ۶۴۰ در نظر گرفته شده است تا حتی در صورت ایجاد ۶۴ goroutine هر یک ۱۰ تسک برای اجرا داشته باشند. در ابتدای تابع تعداد تسک‌های goroutine ها براساس تعدادشان به دست می‌آید. سپس تمام goroutine ها بر اساس نوع تسک تابع مناسب را به تعداد به دست آمده فراخوانی می‌کنند و معیارهای مورد نظر اندازه‌گیری می‌شوند. در نهایت نتایج به دست آمده در داده‌ساختار `Result` برگردانده می‌شوند. با داشتن نتایج توسط توابع `resultToRow` و `saveCSV` آن‌ها را در جدولی در پوشه `results` ذخیره می‌کنیم.

در نهایت تابع `savePlots` صدا زده می‌شود تا از روی نتایج نمودارهای مناسب را رسم و ذخیره کند. در این تابع از کتابخانه خارجی `gonum.org/v1/plot` برای رسم نمودارها استفاده می‌شود. در ابتدا تمام معیارها را تعریف می‌کنیم. سپس به ازای هر معیار آن را به عنوان محور `y` و تعداد goroutine ها را به عنوان محور `x` در نظر می‌گیریم و سه حالت مختلف `GOMAXPROCS` که تعداد هسته‌های پردازشی مورد استفاده از پردازنده را نشان می‌دهد در یک نمودار رسم می‌کنیم.

۳.۳ تحلیل نتایج



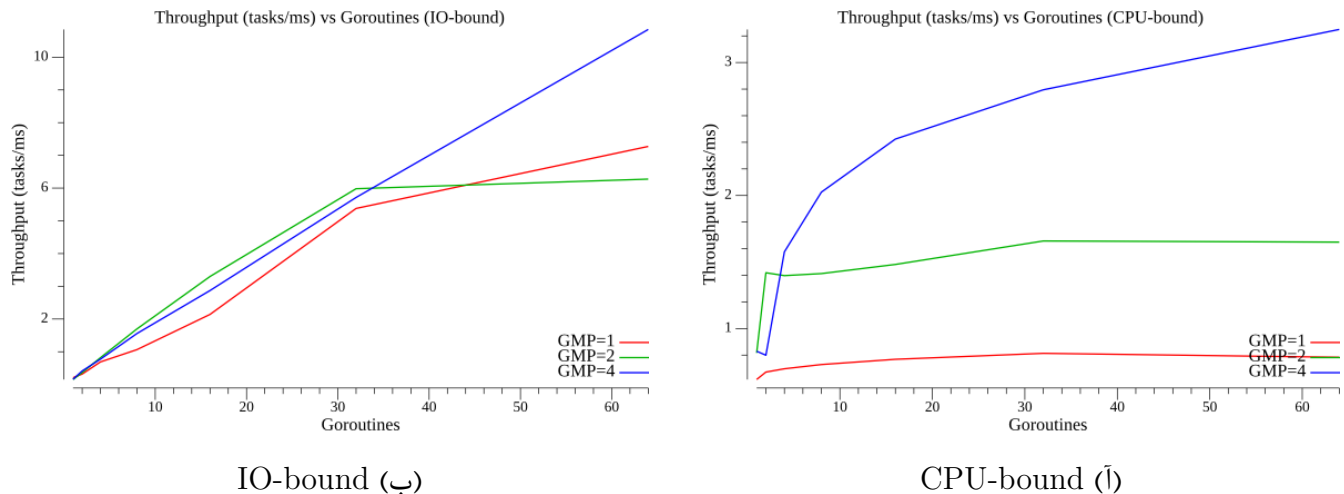
IO-bound (ب)



CPU-bound (آ)

شکل ۱: تغییرات زمان اجرا بر حسب تعداد goroutine برای تسک‌های CPU و IO

در نمودار ۱ نسبت تغییرات زمان اجرا برای تعداد goroutine‌های گوناگون برای هر دو تسک به ازای تعداد هسته‌های پردازشی مختلف نشان داده شده است. در هر دو نوع بار، افزایش تعداد goroutine تا یک نقطه زمان کل را کاهش می‌دهد، اما میزان و چگونگی این کاهش متفاوت است. برای بارهای CPU-bound، وجود هسته‌های بیشتر (افزایش GOMAXPROCS) تأثیر چشمگیری دارد؛ مثلاً با ۱ هسته زمان کل از ۱۰۳۸ میلی‌ثانیه (۱ goroutine) به ۷۸۷ میلی‌ثانیه (۳۲ goroutine) کاهش می‌یابد، در حالی که با ۴ هسته این زمان از ۷۷۲ میلی‌ثانیه (۱ goroutine) به ۱۹۷ میلی‌ثانیه (۶۴ goroutine) می‌رسد. در بارهای IO-bound نیز افزایش هسته‌ها مفید است، اما تأثیر آن کمتر است، زیرا هر تسک الزاماً ۳ میلی‌ثانیه sleep دارد که سهم قابل توجهی از زمان اجرا را تشکیل می‌دهد و با موازی‌سازی بخش محاسباتی نمی‌توان آن را حذف کرد. با این وجود، همچنان با افزایش هسته‌ها و goroutine‌ها همپوشانی بهتری بین خواب و محاسبه ایجاد می‌شود و زمان کل کاهش می‌یابد (مثلاً با ۶۴ goroutine و ۱ هسته ۸۸ میلی‌ثانیه و با ۴ هسته ۵۹ میلی‌ثانیه).



شکل ۲: تغییرات گذردهی بر حسب تعداد goroutine برای تسک‌های CPU و IO

در نمودار ۲ روند گذردهی مشاهده می‌شود. برای بار CPU-bound با ۱ یا ۲ هسته، گذردهی تا حدود ۳۲ goroutine افزایش می‌یابد و سپس ثابت یا اندکی کاهش می‌یابد (مثلاً با $GOMAXPROCS=1$ از ۰.۶۲ به ۰.۸۱ در ۳۲ goroutine رسیده و سپس به ۰.۷۸ در ۶۴ سقوط می‌کند). این رفتار نشانه‌ای غالب شدن سربرار تعویض زمینه (context switching) بر منافع هم‌روندی است. با ۴ هسته، گذردهی به طور پیوسته تا ۶۴ goroutine افزایش می‌یابد (از ۰.۸۳ به ۳.۲۵)، زیرا هسته‌های کافی برای اجرای هم‌زمان تعداد زیادی goroutine بدون تعویض زمینه‌ی سنگین وجود دارد. در بار IO-bound، افزایش goroutine باعث رشد چشمگیر گذردهی می‌شود و این رشد حتی با یک هسته نیز ادامه دارد (از ۰.۲۱ برای ۱ goroutine به ۷.۲۷ برای ۶۴ goroutine). دلیل آن این است که goroutine با sleep با داوطلبانه پردازنده را رها می‌کنند و سربرار تعویض زمینه بسیار کمتر از بار CPU-bound است. با اضافه شدن هسته‌های بیشتر، گذردهی باز هم افزایش می‌یابد (به ۱۰.۸۵ با ۴ هسته)، زیرا بخش محاسباتی می‌تواند روی هسته‌های مختلف به‌طور هم‌زمان اجرا شود و خواب‌ها بهتر همپوشانی شوند.

جدول ۱: مقایسه نتایج برای حالات مختلف

Goroutines/maxprocs	Workload	Total (ms)	Throughput	Avg Lat
1/1	CPU	1038	0.62	1.62
4/1	CPU	918	0.70	4.98
1/1	IO	3087	0.21	4.82
4/1	IO	929	0.69	5.81
1/4	CPU	772	0.83	1.21
4/4	CPU	406	1.58	1.28
1/4	IO	3989	0.16	6.23
4/4	IO	828	0.77	5.16

همان طور که از **جدول ۱** پیداست، افزایش GOMAXPROCS از ۱ به ۴ تقریباً در تمام سناریوها (به جز بار IO-bound با ۱ goroutine که ممکن است تحت تأثیر سربار یا نوسان باشد) زمان کل را کاهش و گذردهی را افزایش می‌دهد. در بار CPU-bound مزیت هسته‌های بیشتر کاملاً مشهود است (مثلاً با ۴ goroutine زمان از ۹۱۸ میلی‌ثانیه به ۴۰۶ میلی‌ثانیه کاهش می‌یابد)، در حالی که در بار IO-bound بهبود نسبی است (از ۹۲۹ به ۸۲۸ میلی‌ثانیه با ۴ goroutine) زیرا بخش sleep را نمی‌توان موازی‌سازی کرد. همچنین مشاهده می‌شود که میانگین تأخیر برای CPU-bound با افزایش هسته‌ها کاهش می‌یابد، اما برای IO-bound تغییر چندانی ندارد و بیشتر تابع ۳ میلی‌ثانیه خواب اجباری است.

در بار CPU-bound اثرات تعویض زمینه (context switching) بیشترین آسیب را به کارایی وارد می‌کند. وقتی تعداد هسته‌ها محدود باشد (۱ یا ۲)، افزایش زیاد goroutine سربار قابل توجهی ایجاد می‌کند و پس از حدود ۳۲ goroutine دیگر بهبودی حاصل نمی‌شود؛ حتی ممکن است کارایی کاهش یابد. این در حالی است که در بار IO-bound به دلیل رها شدن داوطلبانه‌ی پردازنده هنگام sleep، goroutine‌های بیشتر همواره مفید هستند و تعویض زمینه هزینه‌ی کمتری دارد. با ۴ هسته، افزایش تا ۶۴ goroutine همچنان گذردهی را بالا می‌برد، زیرا ظرفیت پردازشی کافی برای اجرای موازی تعداد زیادی از آن‌ها وجود دارد. بنابراین نقطه‌ی بی‌فایده شدن هم‌روندی به نوع بار و منابع سیستم بستگی دارد: برای بار CPU-bound روی ۱-۲ هسته، حدود ۳۲ goroutine بهینه است؛ برای بار IO-bound یا سیستم‌های با هسته‌های زیاد، می‌توان تعداد بیشتری goroutine به کار برد بدون آن که کارایی افت کند.

۴.۳ trace

در ابتدای تابع `main`، `trace` را فعال می‌کنیم بدین ترتیب با اجرای برنامه فایل `trace.out` ایجاد می‌شود و با باز کردن آن در مرورگر اطلاعات گوناگونی در دسترس قرار می‌گیرد.



شکل ۳: وضعیت ریسمان‌ها به دست آمده از `trace`

۴ بخش سوم: ساخت سرویس، کانتینری سازی و اجرا در ماشین مجازی

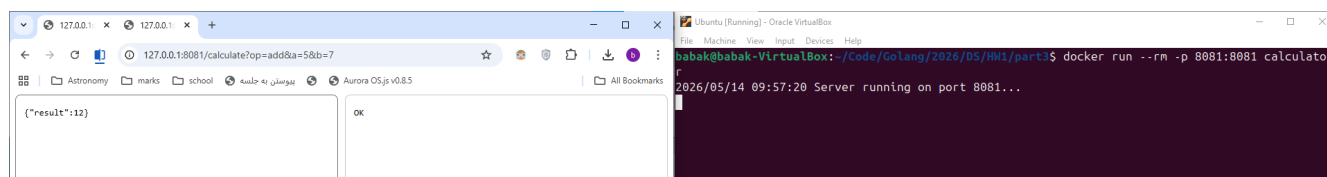
۱.۴ شرح مسئله

برای این بخش قصد داریم برنامه‌ای مشابه ماشین حساب بخش اول ایجاد کنیم و با تبدیل آن به کانتینر آن را با کمک داکر به عنوان سرور بالا بیاوریم. بدین ترتیب می‌توان با ارسال درخواست‌های HTTP محاسبات ریاضیاتی خود را انجام دهد.

۲.۴ پیاده‌سازی

از کدهای Worker از قسمت اول استفاده می‌کنیم تا تابع `calculate` را بنویسیم. تنها تفاوت آن است که در اینجا تابع ورودی‌های `http.Request` و `http.ResponseWriter` را دریافت می‌کند و با استخراج آرگومان از `http.Request` مانند Worker محاسبات را انجام می‌دهد و در در نهایت در `http.ResponseWriter` می‌نویسد. تابع `health` هم که برای اطمینان از بالا بودن سرور نوشته شده است فقط رشته `OK` را برمی‌گرداند. با دریافت درخواست، اگر `endpoint` چیزی به جز `calculate` و `health` بود ارور برمی‌گردانیم و گرنه تابع مربوطه را صدا می‌زنیم و سرور را روی پورت ۸۰۸۱ بالا می‌آوریم.

فایل `Dockerfile` برای ایجاد کانتینر ضروری است. ما از `alpine` برای کم کردن حجم استفاده می‌کنیم. در نهایت کانتینر را ایجاد می‌کنیم و داکر را بالا می‌آوریم. پس از اضافه کردن `Port Forwarding` در `Virtual Box` برای ارزیابی کارکرد درست صرفاً در مرورگر سیستم‌عامل میزبان به پورت ۸۰۸۱ آدرس محلی نمونه‌هایی مشابه سوال اول را می‌دهیم و خروجی را بررسی می‌کنیم.



شکل ۴: اجرای فایل داکر و تست آن در سیستم‌عامل میزبان